Configuração após a clonagem

Guia em português para instalar o projeto, abrir a aplicação e configurar a extração automática no Windows. Preparado para compartilhamento com a equipe. Versão: 8 de novembro de 2026.

Antes de começar: disponibilidade dos scripts

Este guia corresponde à implementação local da branch feat/171-extraction-automation, baseada na main. Na emissão deste PDF, as melhorias de automação ainda aguardavam commit, PR e integração. Uma clonagem da main só terá todos os comandos descritos depois da publicação e integração dessa implementação.

O responsável pela entrega deve informar ao grupo quando a versão estiver publicada. Não use uma branch remota de mesmo nome como garantia: ela pode ainda não conter as alterações locais descritas aqui.

Ordem de execução

1. Instalar os pré-requisitos e clonar o repositório.
2. Preparar o ambiente e escolher o banco.
3. Conferir as variáveis e as migrações.
4. Iniciar backend e frontend; validar o acesso.
5. Executar uma coleta manual e conferir o resultado.
6. Simular e ativar o agendamento diário.
7. Monitorar, atualizar e manter a instalação.

Convenções usadas no documento

Todos os blocos usam Windows PowerShell. Salvo indicação diferente, execute os comandos na raiz datumlex-core, a pasta que contém backend, frontend e scripts. Execute uma linha por vez e avance apenas se ela concluir sem erro.

Os valores em MAIÚSCULAS são exemplos para substituir. O guia não contém senhas reais. A configuração é local: o banco, os arquivos e as credenciais de uma máquina não são baixados ao clonar o código.

Escopo

Este é um procedimento de desenvolvimento e operação local. O servidor de desenvolvimento do Django e o Vite usados aqui não constituem uma implantação de produção para clientes.

1. Pré-requisitos e clonagem

Instale Git, Python 3.12 ou superior e Node.js compatível com o frontend (mínimo documentado no projeto: 22.12). Para PostgreSQL em contêiner, instale também Docker Desktop e conclua a configuração de WSL 2 exigida pelo instalador. Para SQLite, Docker não é necessário.

Downloads oficiais: git-scm.com/downloads/win, python.org/downloads/windows, nodejs.org/en/download e docs.docker.com/desktop/setup/install/windows-install/.

Conferir as ferramentas

```
git --version
py -3 --version
node --version
npm.cmd --version
# Somente para PostgreSQL com Docker:
docker --version
docker compose version
docker info
```

Se py não existir, confira python --version. O setup aceita o executável python como alternativa. Abra um novo terminal após instalar as ferramentas. docker info deve mostrar o serviço do Docker sem erro de conexão.

Clonar a versão publicada

```
git clone https://github.com/DatumLex/datumlex-core.git
cd datumlex-core
git branch --show-current
Test-Path .\scripts\setup.ps1
Test-Path .\backend\scripts\check_etl_setup.py
```

Após a integração, use a main. O último teste confirma a presença do verificador desta implementação e deve retornar True. Se retornar False, confirme com a equipe se as melhorias foram publicadas. Um repositório privado exige uma conta GitHub autorizada.

Se o PowerShell bloquear scripts locais

```
powershell -NoProfile -ExecutionPolicy RemoteSigned
```

Esse comando abre outra sessão. Nela, volte à raiz do projeto e continue. A política vale para esse processo; regras da organização podem exigir outro procedimento. Os comandos usam npm.cmd para evitar o bloqueio do arquivo npm.ps1.

2. Preparar o ambiente e o banco

Etapa comum: instalar sem migrar ainda

```
.\scripts\setup.ps1
notepad .\backend\.env
```

O setup cria backend/.venv, instala os requisitos Python e as dependências do frontend. Também cria backend/.env a partir do exemplo e gera os segredos locais. Se .env já existir, ele é preservado. Não copie novamente o exemplo sobre uma configuração existente.

Para uma máquina dedicada somente à extração, use setup.ps1 -BackendOnly em vez do comando acima. Nesse caso, não é necessário instalar ou executar o frontend.

Caminho A: SQLite, início mais simples

Em backend/.env, deixe DATABASE_URL vazio. Salve o arquivo e execute, na raiz:

```
DATABASE_URL=

.\scripts\setup.ps1 -BackendOnly -Migrate -SkipDependencies
```

As migrações criam as tabelas em backend/data/datumlex_v2.sqlite3. O comando também registra o catálogo de tribunais. Esse caminho não exige Docker nem serviço de PostgreSQL.

Caminho B: PostgreSQL com Docker

No .env, preencha as duas linhas com a mesma senha local. Se houver caracteres especiais, codifique a senha para URL somente em DATABASE_URL. Não use literalmente o valor de exemplo.

```
POSTGRES_PASSWORD=SUA_SENHA_LOCAL
DATABASE_URL=postgresql://datumlex:SUA_SENHA_LOCAL@127.0.0.1:5433/datumlex_v2
```

Abra o Docker Desktop, aguarde o serviço e execute na raiz:

```
docker compose --project-directory backend `
  -f backend/compose.yaml up -d --wait db
.\scripts\setup.ps1 -BackendOnly -Migrate -SkipDependencies
```

Parâmetros: host 127.0.0.1, porta 5433, usuário datumlex e banco datumlex_v2. O volume do Docker mantém os dados. Alterar POSTGRES_PASSWORD não troca a senha de um volume já criado. Trocar de SQLite para PostgreSQL não transfere os registros entre bancos.

3. Conferir as configurações

Todas as variáveis abaixo ficam em backend/.env. Os comandos de instalação e coleta aceitam o banco local configurado. Não coloque chaves no frontend nem envie .env ao GitHub ou ao grupo.

Banco e acesso da aplicação

DATABASE_URL: vazia para SQLite ou conexão PostgreSQL local.
POSTGRES_PASSWORD: senha utilizada pelo Compose no PostgreSQL.
DJANGO_SECRET_KEY: segredo gerado no setup; mantenha privado.
DATUMLEX_SUPPORT_PASSWORD: senha inicial de suporte gerada no setup; guarde em canal privado.

Endereços para desenvolvimento local

```
DJANGO_DEBUG=true
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
CSRF_TRUSTED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Mantenha o frontend na porta 5173 para seguir o guia. Se mudar endereço ou porta, ajuste as origens permitidas e reinicie o backend. DEBUG=true é a configuração local deste procedimento.

Fontes públicas e integrações opcionais

PANGEA_SEARCH_BASE_URL: mantenha o endereço do exemplo para a pesquisa pública do Pangea.
DATAJUD_API_KEY: necessária somente para os comandos separados de DataJud. Obtenha a chave pública atual no endereço oficial indicado no .env.example.
BNP_API_BASE_URL, BNP_TOKEN_URL, BNP_API_CLIENT_ID e BNP_API_CLIENT_SECRET: integração autenticada separada. Mantenha os endereços padrão e deixe credenciais vazias se não usar essa integração. A busca pública do Pangea não depende dessas credenciais.

Validar o Django

```
cd backend
.\.venv\Scripts\python.exe manage.py check
.\.venv\Scripts\python.exe manage.py showmigrations
cd ..
```

Resultado esperado: nenhuma falha no check e migrações aplicadas marcadas com [X]. Uma clonagem não contém o acervo de outra instalação: as migrações preparam o banco, mas os dados precisam ser coletados ou restaurados de um backup compatível.

4. Abrir a aplicação e entrar

Terminal 1: backend

Abra um terminal na raiz. Se escolher PostgreSQL, mantenha o Docker e o banco em execução. Então:

```
cd backend
.\.venv\Scripts\python.exe manage.py runserver 127.0.0.1:8000
```

Deixe esse terminal aberto. Confira no navegador http://127.0.0.1:8000/api/health/. A resposta deve indicar status ok; o campo database depende do banco selecionado.

Terminal 2: frontend

Abra outro terminal na raiz:

```
cd frontend
# Se o setup completo já instalou, não é preciso repetir o ci.
npm.cmd ci
npm.cmd run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

Acesse http://127.0.0.1:5173/. O frontend encaminha as chamadas /api para 127.0.0.1:8000. Nessa configuração local, não é necessário criar .env do frontend.

Primeiro acesso

A tela de login oferece o usuário de suporte suport. A senha inicial corresponde a DATUMLEX_SUPPORT_PASSWORD, definida antes da primeira migração. A conta de suporte existente não tem a senha redefinida apenas por editar o .env.

Usuários comuns podem solicitar cadastro pela interface e aguardam aprovação de um responsável, que define os acessos e tribunais. Não compartilhe a senha de suporte no grupo; cada integrante deve ter seu próprio acesso.

Se a conta de suporte não foi provisionada

Com a configuração revisada e o banco disponível, abra outro terminal na raiz:

```
cd backend
.\.venv\Scripts\python.exe manage.py bootstrap_support
cd ..
```

O comando provisiona a conta de forma idempotente e preserva uma senha já existente. Um dashboard vazio não significa falha de conexão: ainda pode faltar coletar dados.

5. Executar a primeira extração

Na raiz do projeto, comece por um ciclo limitado de TJDFT e Pangea. A coleta não precisa dos servidores de desenvolvimento abertos; precisa do banco configurado e de acesso às fontes.

```
.\scripts\run-etl.ps1 -Once -BatchSize 25 `
  -Sources 'tjdft,pangea'
.\scripts\status.ps1
```

O comando verifica a instalação, coleta material, executa o NLP, importa os resultados e atualiza a cobertura. Preserve os arquivos gerados: eles guardam originais, cursores e evidências do processamento.

Conferir antes de agendar

Procure o diagnóstico de configuração ready e a execução completa com estado completed. Se houver falha, abra os logs indicados por status e resolva antes de ativar o agendamento. Um ciclo pode concluir sem novos registros se os resultados forem duplicados ou não elegíveis.

Incluir todas as fontes configuradas

```
.\scripts\run-etl.ps1 -Once -BatchSize 25 `
  -DoctrineLimit 1 `
  -Sources 'doctrine,pangea,tjdft,ementas,djen'
```

Fontes aceitas: doctrine, pangea, tjdft, ementas e djen. O alcance de cada fonte depende do coletor, da disponibilidade e das regras de seleção. A lista não garante que todo tribunal ou assunto terá novos dados a cada execução.

Entender os parâmetros

-Once: executa um único ciclo.
-BatchSize: lote do TJDFT, múltiplo de 25 entre 25 e 1.000; não limita o total de todas as fontes.
-DoctrineLimit: limite do coletor de doutrina.
-Target: meta acumulada de textos únicos do TJDFT; padrão 20.000.
-Sources: fontes separadas por vírgula, sem espaços.
-Mode process: processa material arquivado sem iniciar a coleta.

```
.\scripts\run-etl.ps1 -Mode process -Once `
  -Sources 'tjdft,pangea'
```

As quantidades variam com duplicatas, elegibilidade e resposta das fontes. As classificações do NLP são evidências candidatas e continuam sujeitas à revisão jurídica.

6. Ativar a automação diária

Ative somente após a execução manual bem-sucedida. O exemplo agenda um ciclo diário às 19h40, no fuso configurado no Windows. Troque o horário e as fontes conforme a decisão da equipe.

Primeiro: simular sem instalar

```
.\scripts\install-schedule.ps1 -At '19:40' `
  -BatchSize 100 -DoctrineLimit 1 `
  -Sources 'doctrine,pangea,tjdft,ementas,djen' -WhatIf
```

Confira o nome da tarefa, o caminho do PowerShell, o script e a pasta de execução. -WhatIf não registra nem executa a tarefa.

Depois: ativar removendo -WhatIf

```
.\scripts\install-schedule.ps1 -At '19:40' `
  -BatchSize 100 -DoctrineLimit 1 `
  -Sources 'doctrine,pangea,tjdft,ementas,djen'
.\scripts\status.ps1
```

O instalador cria uma tarefa no Agendador de Tarefas do Windows. O nome é derivado da pasta do clone. Anote o nome exibido; instalar novamente na mesma pasta atualiza a tarefa. Acrescente -RunNow se desejar iniciar um ciclo imediatamente após instalar.

Condições para executar

Mantenha o computador ligado, o usuário conectado ao Windows e a rede disponível. Se usar PostgreSQL, mantenha o Docker e o banco iniciados. A tarefa usa a sessão interativa do usuário; não é um serviço de servidor que funciona sem login.

Cada disparo executa um ciclo completo com -Mode all -Once. O agendamento usa caminhos absolutos, recupera o horário perdido quando disponível, impede sobreposição da mesma tarefa e possui limite de execução de seis horas. Use somente uma instalação agendada por banco de dados.

Confirmar o primeiro disparo

Após o horário programado, execute status.ps1. Confira a próxima execução, o último resultado da tarefa e o estado do ETL. Abra taskschd.msc, localize o nome informado e confira a ação se precisar investigar. Agendamento instalado não significa que a coleta já foi concluída.

7. Acompanhar e manter

Consultar o estado

```
.\scripts\status.ps1
```

Os arquivos ficam sob backend/data/operations/: etl-all.json registra o ciclo completo; etl-collect.json e etl-process.json registram modos separados; etl-logs/<job>/ contém a saída e os erros de cada etapa; preflight-logs/ guarda os diagnósticos iniciais. collection-coverage.json resume a cobertura observada. Os resultados do NLP ficam sob backend/data/nlp-local/jobs/.

Pausar, reativar ou remover

Substitua NOME_EXATO_DA_TAREFA pelo nome exibido em status. Execute apenas a ação desejada:

```
Disable-ScheduledTask -TaskName 'NOME_EXATO_DA_TAREFA'
Enable-ScheduledTask -TaskName 'NOME_EXATO_DA_TAREFA'
Unregister-ScheduledTask -TaskName 'NOME_EXATO_DA_TAREFA' `
  -Confirm:$false
```

Desativar impede novos disparos; não interrompe necessariamente um ciclo já em andamento. Remover a tarefa não apaga o banco. Para mudar horário ou limites, execute novamente o instalador com os novos valores. Se mover o clone, desative a tarefa antiga e reinstale a partir da nova pasta.

Rotina dos próximos dias

Não recrie .env nem .venv a cada uso. Inicie o banco quando necessário e abra backend e frontend como na seção 4. Para encerrar os servidores de desenvolvimento, use Ctrl+C em cada terminal. Se a automação vai rodar, não desligue o banco necessário a ela.

Atualizar o código publicado

Com trabalho local preservado, agendamento pausado e nenhuma extração em andamento, atualize a branch aprovada pela equipe. O exemplo abaixo pressupõe main e árvore de trabalho limpa:

```
git switch main
git pull --ff-only
.\scripts\setup.ps1 -Migrate
```

Confira as notas da entrega e faça backup antes de migrações. Execute um ciclo manual e reinstale o agendamento para atualizar seus parâmetros; por fim reative a tarefa se ela continuar desabilitada. O setup completo exige Node; em uma máquina só de coleta, acrescente -BackendOnly.

8. Resolver problemas e validar

Falhas mais comuns

Ferramenta não encontrada: verifique PATH e reabra o terminal.
Script ausente: confira a pasta e se a implementação foi publicada.
Docker indisponível: abra Docker Desktop e confira docker info.
Senha PostgreSQL inválida: confira a senha original do volume e DATABASE_URL. Editar o .env não altera a senha interna do banco.
Porta ocupada: verifique se há outra instância usando 5433, 8000 ou 5173.
Frontend sem conexão: teste /api/health/ e mantenha o backend aberto.
Sem dados: confira se houve coleta/importação e consulte a cobertura; migrar não baixa o acervo.
Fonte falhou: consulte o erro da etapa, resolva rede ou configuração e repita o ciclo.
Agendamento não executou: confirme usuário conectado, horário/fuso, computador ligado, tarefa habilitada e banco disponível.

Banco não preparado ou migrações pendentes

```
.\scripts\setup.ps1 -BackendOnly -Migrate -SkipDependencies
.\scripts\status.ps1
```

A automação recusa configurações de banco remoto nesta implementação. Para um banco antigo ou restaurado, valide a compatibilidade com a equipe antes de migrar; o procedimento principal foi pensado para uma instalação nova.

Preservação dos dados

Não use docker compose down -v para encerrar o ambiente: a opção -v remove volumes. Não apague backend/data para resolver falhas. Para copiar um acervo, solicite um backup compatível e siga o procedimento de restauração em banco separado descrito em INSTALLATION-GUIDE.md. Código clonado e imagem do PostgreSQL não incluem os dados da equipe.

Checklist de conclusão

[ ] Versão com automação publicada e disponível no clone.
[ ] Ambiente instalado; .env privado e banco escolhido.
[ ] Migrações aplicadas e catálogo de tribunais registrado.
[ ] Backend saudável e frontend acessível, se utilizado.
[ ] Acesso individual testado.
[ ] Ciclo manual concluído e cobertura conferida.
[ ] Agendamento instalado e primeiro disparo verificado.
[ ] Responsável pelo monitoramento e backup definido.

Referências do repositório: README.md; INSTALLATION-GUIDE.md; docs/operations/AUTOMACAO-LOCAL.md; docs/operations/ETL-ELT-LOCAL.md; backend/.env.example; backend/compose.yaml. Este PDF descreve a implementação em preparação, e não substitui a validação da versão publicada.